Variabili, tipi di dato e operatori

Lezione 1 — 3MAK, SST-INF-01c — Fondamenti di programmazione in C

17/09/2026

1 Che cos'è una variabile?

Un microcontrollore PIC ha decine di **registri** interni: ogni casella di un registro è un ***flip-flop*** che mantiene memorizzato un **bit** (0 o 1) e, in base a come viene impostata quella **sequenza di bit**, decide come si deve comportare il microcontrollore (es. la velocità di esecuzione del codice, quale oscillatore usare, se abilitare o meno un watchdog). Un esempio reale è il **registro di configurazione** del PIC16F628A, che raccoglie bit come `FOSC2:0` (tipo di oscillatore), `WDTE` (watchdog abilitato/disabilitato), `PWRTE`, `MCLRE`, `BOREN`, `CP`, `CPD`, `LVP`: il **datasheet** del componente (che può arrivare a 800 pagine) spiega il significato di ciascun bit.

Nota

Il nome che si dà a un registro è concettualmente **equivalente** al nome di una **variabile**: proprio come una variabile, un registro è uno spazio – in questo caso fatto di singoli bit – a cui diamo un nome, e il cui contenuto configura il comportamento della macchina. Programmare a questo livello richiede di conoscere **ogni singolo registro**. **Arduino**, al contrario, **nasconde** questa complessità – comodo, ma limitato. Per lavorare davvero con un PIC serve invece capire i concetti di base della programmazione, a partire dalle **variabili**.

Definizione

Una **variabile** è un **contenitore di dati**: uno spazio in memoria a cui diamo un **nome**, dentro cui possiamo mettere un valore che può cambiare (da qui il nome "variabile").

Esempio

Immagina un **cassetto** nell'armadio di un allievo, usato per contenere le sue magliette. Il cassetto ha un'etichetta con un nome (es. `Gertrude`) e contiene un numero (es. 10 magliette). Se l'allievo ne compra altre, il numero dentro il cassetto **cambia** – ma il cassetto, con il suo nome, **resta lo stesso**. Il cassetto è la **variabile**; il nome è il suo identificatore; il contenuto è il valore che rappresenta.

2 Quanto è grande il cassetto? I tipi di dato

Non tutti i "cassetti" hanno la stessa dimensione: dipende da cosa devono contenere. In **C**, questo si chiama **tipo di dato**.

Definizione

- **char**: 1 **byte** (8 bit) $\rightarrow 2^8 = 256$ combinazioni diverse, da 0 a 255.

- **int**: 2 byte (16 bit) $\rightarrow 2^{16} = 65536$ combinazioni, da 0 a 65535.
- **float**: 32 bit, **con la virgola** $\rightarrow$ circa 4 miliardi di combinazioni, ma usate per rappresentare numeri **decimali** (non solo interi).

Esempio

Se devo memorizzare il numero di magliette in un cassetto (poche decine), basta un **`char`**. Se devo memorizzare il numero di allievi di un intero istituto (centinaia), serve un **`int`**. Se devo memorizzare un saldo in euro con i centesimi (es. 40.322,72), serve un **`float`**, perché `char` e `int` rappresentano solo numeri interi, senza virgola.

Nota

Anche i computer moderni hanno un limite fisico alla memoria che possono gestire, legato al numero di bit dell'architettura: un'architettura a **32 bit** può indirizzare al massimo 2^{32} posizioni di memoria (circa 4 miliardi di byte, cioè **4 GB**); un'architettura a **64 bit** arriva a 2^{64}, un numero enormemente più grande. È il motivo per cui i computer sono passati da 32 a 64 bit.

Attenzione

Un **`float`** **approssima** i numeri con la virgola: non li rappresenta sempre con precisione esatta, per come sono codificati internamente in **binario**. È un dettaglio da tenere presente quando si fanno calcoli delicati (es. finanziari), ma per i nostri scopi il compromesso è più che accettabile.

Esempio

Cosa succede se si prova a mettere un numero **con la virgola** dentro un contenitore **intero**?

```
int p = 12.5;
```

La variabile `p` non contiene `13` (non viene **arrotondata**), ma **`12`**: essendo `p` di tipo `int`, la macchina non è in grado di rappresentare la **parte decimale** in quel contesto e la **tronca** semplicemente, **perdendo** quel dettaglio – non lo arrotonda al più vicino.

3 Dichiarare una variabile in C

Esempio

```
int Gertrude = 10;
```

Questa riga **crea** una variabile di tipo `int`, chiamata `Gertrude`, e vi mette dentro il valore 10. Ogni istruzione in C termina con un **punto e virgola** (`;`). Il tipo e il nome della variabile vanno sempre **separati da uno spazio**, perché il **compilatore** deve poter distinguere le singole parole (i ***token***) del codice.

4 Operatori: fare calcoli con le variabili

Definizione

Un **operatore** è un simbolo che esegue un calcolo o un **confronto** tra due (o più) valori.

Esempio

```
int a = 7;
int b = 5;
int c = a + b;   // c vale 12
```

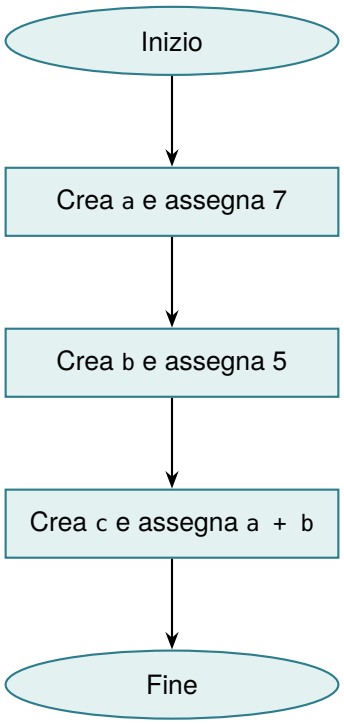


Figura 1: Diagramma di flusso della sequenza che calcola `c = a + b`.

4.1 Operatori aritmetici

- `+` somma, `-` sottrazione, `*` **moltiplicazione** (asterisco, non "x"), `/` divisione.
- `%` **modulo**: calcola il ***resto*** di una divisione tra **interi**.

Esempio

$5/2$ (divisione intera) fa 2, con resto 1. Quindi: `5 % 2` vale 1. Il modulo è utilissimo per verificare se un numero è **pari o dispari**:

```
if (x % 2 == 0) {
  // x e' pari
}
```

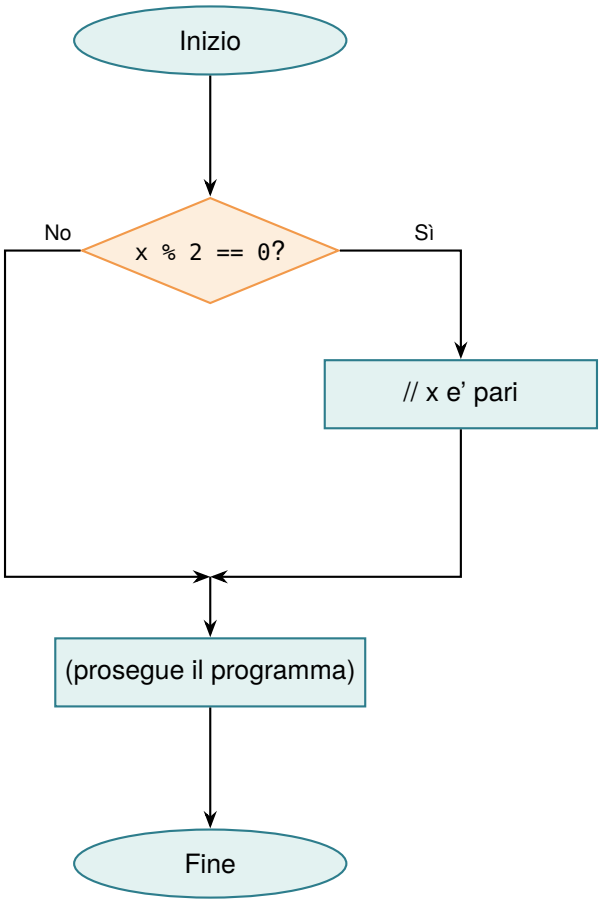


Figura 2: Diagramma di flusso dell'`if` che verifica se `x` è pari: se il resto della divisione per 2 è zero si esegue il blocco, altrimenti si prosegue direttamente.

4.2 Operatori di confronto

Attenzione

L'errore più comune di chi inizia a programmare: **confondere** `=` con `==`.

- `a = b` è un **operatore di assegnazione**: **copia** il valore di `b` dentro `a`. È sempre un'azione, non una domanda.
- `a == b` è un **operatore di confronto**: **chiede** "`a` è uguale a `b`?" e risponde **vero o falso**, senza modificare nulla.

Scrivere `if (a = b)` invece di `if (a == b)` è un **errore classico**: non confronta nulla, **assegna** `b` ad `a` e valuta il risultato come condizione – quasi mai quello che si voleva fare.

Esempio

Se `a` è un cassetto con 4 magliette e scrivo `a = b`, prendo il contenuto di `b` e lo **copio** dentro `a` – non lo sposto, il valore originale in `b` **resta intatto**.

Gli altri **operatori di confronto** sono: `>`, `<`, `>=`, `<=` (maggiore, minore, maggiore-uguale, minore-uguale) e `!=` (**diverso**).

4.3 Operatori logici

Definizione

Gli **operatori logici** combinano condizioni **vero/falso**, esattamente come le porte logiche in elettronica digitale:

- `&&` (**AND**, "e commerciale doppio"): vero solo se **entrambe** le condizioni sono vere.
- `||` (**OR**, doppia barra verticale): vero se **almeno una** condizione è vera.
- `!` (**NOT**): **inverte** il valore (vero diventa falso e viceversa).

Nota

In C non esiste un vero e proprio tipo **booleano** nativo (in molti compilatori): per convenzione, il valore **0 rappresenta falso**, e qualunque valore **diverso da 0** rappresenta **vero** (anche numeri negativi). Alcuni programmatori usano le costanti `TRUE`/`FALSE` definite a parte per rendere il codice più leggibile.

Esempio

Se `a = 7` e `b = 5`, entrambi sono "veri" (**diversi da 0**): `a && b` vale **vero**. Se cambio `b = -3`, è ancora diverso da 0, quindi ancora vero: `a && b` resta vero. Solo se una variabile vale esattamente `0` viene considerata **falsa**.

5 Cosa ricordare di questa lezione

- Una **variabile** è un **contenitore** nominato in memoria, il cui **valore** può cambiare.
- I **tipi di dato** (char, int, float) determinano quanto **spazio** occupa una variabile e che **tipo di numeri** può contenere.
- **char** = 1 byte (0-255), **int** = 2 byte (0-65535), **float** = numeri **con la virgola**.
- **Non confondere mai** = (assegnazione, copia un valore) con == (confronto, chiede se sono uguali).
- Gli **operatori** si dividono in **aritmetici** (`+ - * / %`), di **confronto** (`== != > < >= <=`) e **logici** (`&& || !`).
- In C, **0 è falso**, **qualunque altro valore** è vero.

Esempio

Da fare per la prossima lezione: scrivi su carta il codice C per dichiarare due variabili intere, sommarle, e verificare se il risultato è **pari o dispari** usando l'operatore **modulo**.